Labortory 2

Spatial Filtering and Thresholding

ECE 7410: Image Processing

July 2nd 2025

Group Member	Student ID
Keegan Rolls	202004149
Nicholas Philpott	202018248

Spatial Filtering

Smoothing

1. *Download the test images (`img1.png` and `img2.png`) from Brightspace under Lab 2.*


2. *Read `img1.png`, convert it to a grayscale image, and display.*

   See the following image `img1_grey.png`

3. *Develop and display a function that will perform spatial filtering (i.e., convolution) on an $M \times N$ pixel image using a general $m \times n$ kernel. The function should take an image and a kernel as inputs and return the filtered image. Do **NOT** use internal functions for this (e.g., MATLAB's **imfilter** function). (**hint:** zero padding the edges of the image is required).*

4. *Filter `img1.png` with each of the five filters below (Kernel 1 to Kernel 5) using your function and display the outputs:*

   1. `Kernel_1` $= \left(\frac{1}{9}\right) *$`ones`$(0)$

      See Figure 1 below.

      `img1.png` with `Kernel_1` applied

      Figure 1: `img1.png` with `Kernel_1` applied

   2. `Kernel_2` $= \left(\frac{1}{49}\right) *$`ones`$(7)$

      See Figure 2 below.

      `img1.png` with `Kernel_2` applied

      Figure 2: `img1.png` with `Kernel_2` applied

   3. `Kernel_3` $=$ `fspecial`$(\text{'average'}, [7, 7])$

      See Figure 3 below.

      `img1.png` with `Kernel_3` applied

      Figure 3: `img1.png` with `Kernel_3` applied

   4. `Kernel_4` $=$ `fspecial`$(\text{'gaussian'}, [3, 3], 0.5)$

      See Figure 4 below.

   5. `Kernel_5` $=$ `fspecial`$(\text{'gaussian'}, [7, 7], 1.2)$

      See Figure 5 below.

5. *Develop and display a function that performs median filtering. A median filter will replace the pixel intensity with the median intensity of the pixels overlapped by the kernel. The function should take the image and the size of the kernel as inputs. (**hint:** zero padding the edges of the image is required).*

`img1.png` with `Kernel_4` applied

Figure 4: `img1.png` with `Kernel_4` applied

`img1.png` with `Kernel_5` applied

Figure 5: `img1.png` with `Kernel_5` applied

6. *Apply the following two median filters to `img1.png` and display the outputs:*
    1. Median filter with kernel size 3×3

       See Figure 6 below.

       `img1.png` with median filter 3×3 applied

       Figure 6: `img1.png` with median filter 3×3 applied

    2. Median filter with kernel size 5×5

       See Figure 7 below.
7. *Compare and discuss the smoothing effects obtained by `Kernel_1` to `Kernel_5` and the two median filters.*

   ...
8. *Discuss the effect the smoothing kernel size has on the output images in your experiments.*

   ...

Sharpening

1. *Use the function you developed in **Smoothing: Step 3** and apply the following three sharpening filters (`Kernel_6` to `Kernel_8`) to `img2.png`.*
    1. $\texttt{Kernel\_6} = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & -2 \\ -1 & 0 & 1 \end{bmatrix}$

       See Figure 8 below.
    2. $\texttt{Kernel\_7} = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$

       See Figure 9 below.
    3. $\texttt{Kernel\_8} = \texttt{fspecial}(\text{'log'}, 3)$

       See Figure 10 below.
2. *Compare and discuss the outputs obtained by `Kernel_6` to `Kernel_8`*

   ...

`img1.png` with median filter 5×5 applied

Figure 7: `img1.png` with median filter 5×5 applied

`img2.png` with `Kernel_6` applied

Figure 8: `img2.png` with `Kernel_6` applied

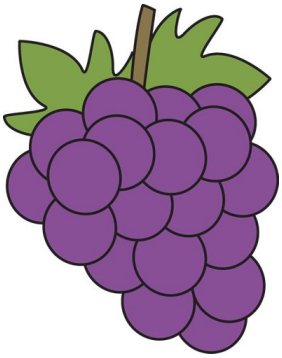
Thresholding

The following equations are from the lab manual.

$$\omega_0(t) = \sum_{i=0}^{t-1} p(i) \quad \mu_0(t) = \sum_{i=0}^{t-1} ip(i)/\omega_0 \quad \omega_1(t) = \sum_{i=t}^{L-1} p(i) \quad \mu_1(t) = \sum_{i=t}^{L-1} ip(i)/\omega_1 \tag{1}$$

$$\sigma_b^2(t) = \omega_1\omega_2 \left[\mu_0 - \mu_1\right]^2 \tag{2}$$

1. *Download the test images (`img3.png` and `img4.png`) from Brightspace under Lab 2 and save them in your working directory.*



2. *Develop and display code to calculate the class probabilities and class means, as well as the inter-class variance (equations 1 and 2) as a complete function to calculate the optimal threshold value for any given image.*

```
%% Q2: Function to calculate optimal threshold using Otsu`s method
function optimal_threshold = calculate_optimal_threshold(img)
    % Convert to grayscale if needed
    if size(img, 3) == 3
        img = rgb2gray(img);
    end

    % Convert to double for calculations
    img = double(img);

    % Calculate histogram
    [counts, ~] = hist(img(:), 0:255);

    % Calculate probabilities
    p = counts / sum(counts);

    % Initialize variables
```

`img2.png` with `Kernel_7` applied

Figure 9: `img2.png` with `Kernel_7` applied

`img2.png` with `Kernel_8` applied

Figure 10: `img2.png` with `Kernel_8` applied

```
    L = 256; % Number of gray levels (0-255)
    max_variance = 0;
    optimal_threshold = 0;

    % Try all possible thresholds
    for t = 1:(L - 1)
        % Calculate class probabilities
        omega_0 = sum(p(1:t));
        omega_1 = sum(p(t + 1:L));

        % Skip if one class is empty
        if omega_0 == 0 || omega_1 == 0
            continue;
        end

        % Calculate class means
        mu_0 = sum((0:(t - 1)) .* p(1:t)) / omega_0;
        mu_1 = sum((t:(L - 1)) .* p(t + 1:L)) / omega_1;

        % Calculate inter-class variance
        sigma_b_squared = omega_0 * omega_1 * (mu_0 - mu_1) ^ 2;

        % Update optimal threshold if this variance is larger
        if sigma_b_squared > max_variance
            max_variance = sigma_b_squared;
            optimal_threshold = t - 1; % Convert to 0-255 range
        end
    end
end
```

3. *Calculate and report the optimal threshold values for* ***`img3.png`*** *and* ***`img4.png`*** *found using your code in* ***Thresholding: Step 2****.*

   - `img3.png`:
     - ...
   - `img4.png`:
     - ...

4. *Develop code to convert a grayscale image to a binary image using a given threshold value. Do* ***NOT*** *use internal functions for this (e.g., the MATLAB function* ***im2bw****).*

5. *Use the optimal thresholds to generate and display binary image for* ***`img3.png`*** *and* ***`img4.png`****.*

   See Figure 11 and Figure 12 below.

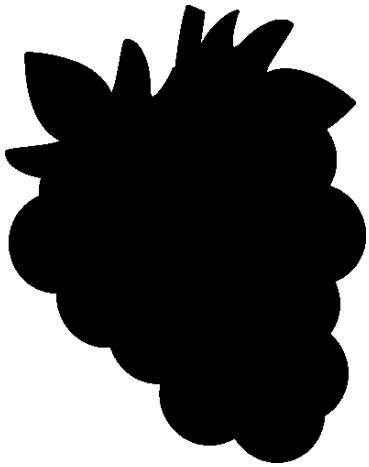


Figure 11: binary image of `img3.png` with $threshold = 177$

Figure 12: binary image of `img4.png` with $threshold = 129$